

Теория формальных языков

Лабораторная работа №1

Вариант 8

Работу выполнила:

Студент группы ИУ9-51Б

Григорьева Софья

Содержание

1	Задание	2
2	Исследование SRS	3
2.1	Завершимость	3
2.2	Классы эквивалентности	3
2.3	Локальная конfluenceнтность	4
2.4	Пополняемость по Кнуту–Бендиксу	4
2.5	Построение SRS $\tilde{\mathcal{T}}'$	6
3	Тестирование	8
3.1	Фазз-тестирование эквивалентности	8
3.2	Метаморфное тестирование	8

1 Задание

Дана SRS:

$arbc \rightarrow caqdbarbar$
 $raqd \rightarrow daqdbarbar$
 $ccrp \rightarrow adaqdqa$
 $dpr \rightarrow prp$

По имеющейся SRS определить:

- завершимость;
- конечность классов эквивалентности по НФ (для построения эквивалентностей считаем, что правила могут применяться в обе стороны). Если их конечное число, то построить минимальную систему переписывания, соответствующую им;
- локальную конfluence-ность и пополняемость по Кнуду–Бендиксу.

По SRS $\mathcal{T}$ тем самым (исключая случай, когда она сразу локально конfluence-тна или конечна и минимальна) строится другая SRS $\mathcal{T}'$, которая должна сохранять те же классы эквивалентности. Если исходная SRS завершима, то правила в $\mathcal{T}'$ должны удовлетворять условию убывания левой части относительно правой по выбранному фундированному порядку $\preceq$.

Провести автоматическое тестирование предполагаемой эквивалентности указанных SRS.

Фазз-тестирование эквивалентности: строится случайное слово ω и случайная цепочка переписываний его в ω' по $\mathcal{T}$. Проверить, можно ли получить ω' из ω (или наоборот) в рамках правил $\mathcal{T}'$.

Метаморфное тестирование: выбрать инварианты, которые должны сохраняться (либо монотонно изменяться) при переписывании в рамках $\mathcal{T}$. Породить случайную цепочку переписываний над случайным словом в $\mathcal{T}'$ и проверить выполнение инвариантов. Как минимум два разных инварианта.

2 Исследование SRS

2.1 Завершимость

Исходная SRS $\mathcal{T}$ является завершаемой.

Доказательство:

Рассмотрим инвариант

$$I(S) = \sum_{i=1}^n g(s_i) 2^{i-1}, \quad \text{где} \quad g(x) = \begin{cases} 10, & \text{если } x = c, \\ 1, & \text{если } x = d, \\ 0, & \text{иначе} \end{cases}$$

Здесь s_i — буква слова на i -й позиции (нумерация начинается с 1).

Проверим инвариант для правил переписывания. Пусть правило применяется к подстроке, первый символ которой находится на m -й позиции в строке ($1 \leq m \leq n$).

I. $apbc \rightarrow caqdbapbar$

$$I(S') = I(S) - 62 \cdot 2^{m-1}$$

II. $paqd \rightarrow daqdbapbar$

$$I(S') = I(S) + 2^{m-1}$$

III. $ccpp \rightarrow adaqdqa$

$$I(S') = I(S) - 12 \cdot 2^{m-1}$$

IV. $dpl \rightarrow pdp$

$$I(S') = I(S) - 3 \cdot 2^{m-1}$$

Получаем убывающий инвариант для правил I, III и IV.

2.2 Классы эквивалентности

Исходная SRS $\mathcal{T}$ имеет бесконечное количество классов эквивалентности. Например, строка вида a^n образует отдельный класс эквивалентности для каждого $n \in \mathbb{N}$.

2.3 Локальная конфлюэнтность

SRS $\mathcal{T}$ не является локально конфлюэнтной, так как слово $arbcscr$ за один шаг может быть преобразовано в слова $caqdbarbarscr$ (по правилу $arbc \rightarrow caqdbarbar$) и $arbadadaqda$ (по правилу $scr \rightarrow adadaqda$), но эти слова являются различными НФ.

2.4 Пополняемость по Книту–Бендиксу

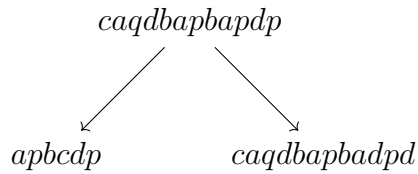
Для применения алгоритма Кнута–Бендикса введём *фундированный порядок* — length-lexicographic order $\prec$ — и, в соответствии с ним, переориентируем правила. (Такое преобразование сохраняет классы эквивалентности исходной SRS $\mathcal{T}$.)

После переориентации система переписываний примет вид:

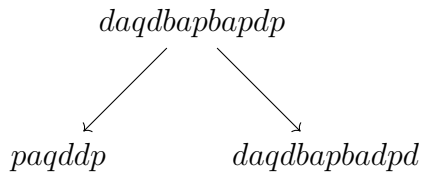
$caqdbarbar \rightarrow arbc$
 $daqdbarbar \rightarrow paqda$
 $adadaqda \rightarrow scr$
 $pdp \rightarrow dpd$

Пополнение системы.

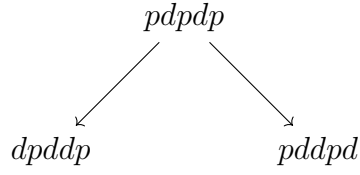
1-я итерация.



Добавляем правило:
 $caqdbarbadpd \rightarrow arbcdp$



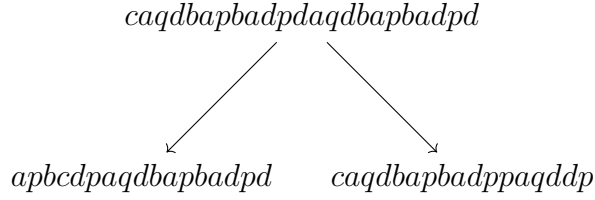
Добавляем правило:
 $daqdbarbadpd \rightarrow paqddp$



Добавляем правило:
 $pddpd \rightarrow dpddp$

Заметим, что на данном этапе ни одно правило нельзя редуцировать.

2-я итерация.



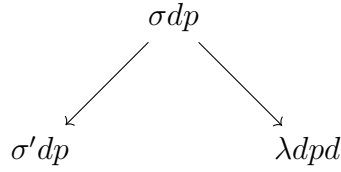
Добавляем правило:
 $caqdbapbadppaqddp \rightarrow apbcdpaqdbapbadpd$

Для практических целей остановимся на разумной длине правил. Добавим в систему правила из первой итерации, поскольку дальнейшее пополнение приводит к росту длины слов, а наша цель — построить эквивалентную SRS, удобную для применения. Кроме того, можно доказать, что система бесконечно пополняется.

Доказательство:

Пусть в системе существует правило, левая часть которого σ оканчивается на букву p , но не на pp , а правая часть σ' не оканчивается на p . Обозначим через λ префикс слова σ , исключая последнюю букву p . Тогда такое слово образует критическую пару с правилом pdp . (*)

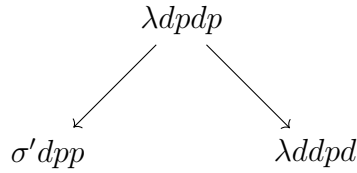
1-я итерация.



Так как правила системы либо уменьшают длину слова, либо сохраняют её, но уменьшают лексикографический порядок $\lambda \succ \sigma'$ и $\lambda dpd \succ \sigma' dp$, поэтому в систему добавляется правило $\lambda dpd \rightarrow \sigma' dp$.

Новое правило снова образует критическую пару с pdp . (*)

2-я итерация.



Аналогично, в систему добавляется правило $\lambda ddpd \rightarrow \sigma' dpp$ и оно снова образует критическую пару с pdp . (*)

Повторяя этот процесс, получаем бесконечную цепочку новых правил.

(*) Образование новых критических пар и правил объясняется тем, что правила системы не могут редуцировать dp^+ на конце слова, а также последовательность букв d , если перед ней не стоит буква p , поэтому результат применения правил на схемах - различные нормальные формы.

В исходной системе присутствует правило $caqdbarbar \rightarrow arbc$, левая часть которого оканчивается на p , но не на pp , а правая часть не оканчивается на p . Следовательно, система SRS $\tilde{T}$ бесконечно пополняется.

2.5 Построение SRS $\tilde{T}'$

На основе исследования SRS $\tilde{T}$ была построена эквивалентная ей SRS $\tilde{T}'$:

$$caqdbarbar \rightarrow arbc$$

$daqdbapbap \rightarrow paqd$
 $adaqdqa \rightarrow ccpp$
 $pdp \rightarrow dpd$
 $caqdbapbadpd \rightarrow apbcdp$
 $daqdbapbadpd \rightarrow paqddp$
 $pddpd \rightarrow dpddp$

3 Тестирование

3.1 Фазз-тестирование эквивалентности

Исходный код программы представлен в файле «fuzzTest.erl».

В программе реализованы следующие основные функции:

- `originalSRS()` – возвращает список исходных правил;
- `equalSRS()` – возвращает список правил, эквивалентных исходным;
- `generateRandomWord()` – генерирует случайное слово длиной от 10 до 30 символов из алфавита "abcdpq";
- `findOccurrences(Sub, Word)` – находит все позиции вхождения подстроки `Sub` в строке `Word`;
- `randomlyTransform(Word, Rules)` – случайным образом применяет правила к слову;
- `getNext(Word, Rules)` – возвращает все возможные варианты слов, которые могут быть получены из `Word` применением одного правила;
- `findTransformOptions(StartWord, Rules)` – находит все слова, достижимые из `StartWord` при последовательном применении правил;
- `areEqual(Small, Big)` – проверяет эквивалентность двух слов;
- `test(TestsNumber)` – выполняет указанное количество фазз-тестов и выводит их результаты.

Программа последовательно генерирует случайные слова, применяет к ним исходные правила, и проверяет эквивалентность слов, через нахождение совпадений между результатами применения эквивалентной системы правил к этим словам, и выводит отчет о каждом тесте.

3.2 Метаморфное тестирование

В качестве инвариантов для метаморфного тестирования были выбраны:

1. сохранение чётности количества букв *c*
2. сохранение чётности суммы количеств букв *b* и *q*

Исходный код программы представлен в файле «metaTest.erl». В программе реализованы следующие основные функции:

- `originalSRS()` – возвращает список исходных правил;
- `equalSRS()` – возвращает список правил, эквивалентных исходным;
- `generateRandomWord()` – генерирует случайное слово длиной от 10 до 30 символов из алфавита "abcdpq";
- `findOccurrences(Sub, Word)` – находит все позиции вхождений подстроки `Sub` в строке `Word`;
- `randomlyTransform(Word, Rules)` – случайным образом применяет правила к слову;
- `isValidInvariant1(Word, WordAfterOriginal, WordAfterEqual)` – проверяет сохранение первого инварианта (чётность количества букв *c*);
- `isValidInvariant2(Word, WordAfterOriginal, WordAfterEqual)` – проверяет сохранение второго инварианта (чётность суммы количеств букв *b* и *q*);
- `test(TestsNumber)` – выполняет указанное количество метаморфных тестов и выводит результаты.

Программа генерирует случайные слова, применяет к ним исходную и эквивалентную системы правил, а затем проверяет, сохраняются ли заданные инварианты после преобразований и выводит отчет о каждом тесте.